

AI SDLC STUDIO

Codeflow & Architecture Documentation

An autonomous, multi-agent AI engine for the Software Development Life Cycle

Version 2.2.0 | Python / FastAPI / LangGraph + React / TypeScript

Generated documentation — reverse-engineered from project source (AI-SDLC-Studio-version-3.zip)

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 Tech Stack	3
2. High-Level System Architecture	4
2.1 Request/Response Pattern.....	4
2.2 Project Phase State Machine.....	4
3. Backend Code Organisation	5
4. Data Model	7
5. Phase 1 — Requirement Agent Codeflow.....	8
5.1 Four-Stage Gated Pipeline	8
5.2 Key Nodes	9
6. Phase 2 — Design Agent Codeflow	11
6.1 Linear Design Pipeline (Stages 1-9)	11
6.2 What Each Specialist Produces.....	12
7. Phase 3 — Development Agent Codeflow	14
7.1 Pipeline Stages	14
7.2 Task Distribution Engine — Parallel Sub-Agent Scheduling	15
7.3 Supporting Core Engines.....	15
8. API Endpoint Reference.....	17
8.1 Projects.....	17
8.2 Requirement Agent (Phase 1)	17
8.3 Design Agent (Phase 2)	17
8.4 Development Agent (Phase 3).....	18
9. Frontend Architecture	19
10. End-to-End Walkthrough	20
11. Running the Project Locally	21
11.1 Backend.....	21
11.2 Frontend.....	21
Appendix A — Source File Tree	22

1. Introduction

AI SDLC Studio is a full-stack application that automates the front half of the software development life cycle. Given a raw problem statement or an uploaded requirements document, it walks a project through three sequential, human-gated AI agent phases:

- Phase 1 — Requirement Agent: turns a problem statement into an IEEE- 830- style Software Requirements Specification (SRS).
- Phase 2 — Design Agent: turns the approved SRS into a System Design Document (SDD) with architecture diagrams, database design, API design and Architecture Decision Records (ADRs).
- Phase 3 — Development Agent: turns the approved SDD into actual generated source code (backend, frontend, database scripts, integration glue, docs), produced by five specialised sub-agents coordinated by an orchestrator.

Every phase is implemented as a LangGraph state machine with explicit human-in-the-loop approval nodes, so a human reviewer must approve (or reject and trigger a regeneration) before the project can move to the next phase. This document describes how the code is organised and how control and data flow through the system.

1.1 Tech Stack

Layer	Technology
Frontend framework	React 18 + Vite 5 + TypeScript
Frontend styling	Tailwind CSS, Lucide React icons
Diagrams (client)	Mermaid.js 10 (renders the SDD's diagrams in-browser)
Backend framework	Python 3.10+, FastAPI, Uvicorn
Agent orchestration	LangGraph (StateGraph) + LangChain
LLM provider	Google GenAI (Gemini) via a custom provider, with a deterministic Mock LLM fallback
Persistence	SQLAlchemy 2.0 ORM over SQLite (dev) / PostgreSQL (prod)
Agent checkpointing	LangGraph SqliteSaver (separate sdlc_checkpoints.db)
Document export	ReportLab (PDF), python-docx (Word), Markdown, raw JSON
Auth	Static API key via X-API-Key header (or api_key query param)

2. High-Level System Architecture

The system follows a classic three-tier shape (React client → FastAPI service → relational database), with the AI reasoning implemented as three LangGraph agents sitting behind the FastAPI layer. Every agent call is synchronous from the frontend's point of view: the frontend calls a REST endpoint, the backend drives the LangGraph graph forward until it hits either an interrupt (human approval gate) or the END node, persists the resulting state, and returns JSON.

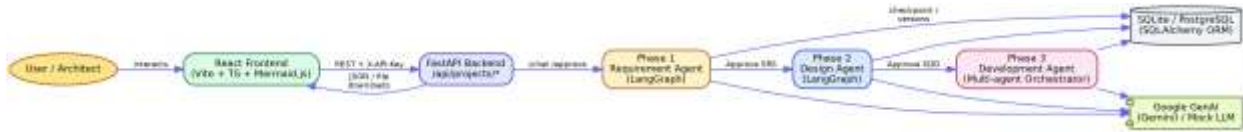


Figure 1 — End-to-end system overview: client, API, three-phase agent pipeline, storage and LLM provider.

2.1 Request/Response Pattern

Every backend route (except `/`, `/health`, `/docs`, `/openapi.json` and `/redoc`) is protected by the `verify_api_key` dependency in `main.py`, which checks the `X-API-Key` header (or `api_key` query parameter) against `STUDIO_API_KEY` from the environment. All business routes live under `/api/projects` and are grouped by phase: project management, Requirement Agent, Design Agent and Development Agent.

2.2 Project Phase State Machine

A Project row (see `models.Project`) carries two top-level fields that drive the UI: `current_phase` (REQUIREMENT → DESIGN → DEVELOPMENT) and `status` (IN_PROGRESS, AWAITING_APPROVAL, APPROVED, REJECTED). These are updated by the service layer as each LangGraph agent progresses or is approved/rejected, and are what `GET /api/projects/{id}/status` reports back to the frontend dashboard.

3. Backend Code Organisation

The backend lives under backend/app. Responsibilities are layered: FastAPI routes in main.py never talk to LangGraph directly — they call into a service module, which owns DB reads/writes and drives the relevant agent graph.

Path	Responsibility
app/main.py	FastAPI app, CORS, API-key auth dependency, all route handlers
app/database.py	SQLAlchemy engine/session factory + shared LangGraph SqliteSaver checkpoint
app/models.py	SQLAlchemy ORM models (Project, Requirement*, DesignDocument*, Development*, Epic/Feature/UserStory, ProjectAgentState, ...)
app/schemas.py	Pydantic request/response schemas (ProjectCreate, ChatRequest, RequirementMemory, ProjectDocument, SDDOutput, HumanReviewSubmit, ...)
app/agents/requirement_agent.py	Phase 1 LangGraph graph: builds the SRS through a 4-stage gated pipeline
app/agents/design_agent.py	Phase 2 LangGraph graph: builds the SDD (architecture, DB, API, ADRs, diagrams)
app/agents/development/orchestrator.py	Phase 3 LangGraph graph: coordinates 5 sub-agents to generate source code
app/agents/development/agents/*.py	The 5 Phase-3 worker agents: database, backend, frontend, api_integration, documentation
app/agents/development/core/*.py	Supporting engines: rules_engine, template_engine, prompt_manager, merge_validation, memory_manager, artifact_generator, code_snippet_store
app/agents/development/planning_engine.py	Builds the module/task plan and dependency graph consumed by project_planner
app/agents/development/context_builder.py	Builds development_context from the approved SRS + SDD
app/services/project_service.py	Project CRUD, SRS version helpers, activity log, completeness calculations
app/services/agent_service.py	Drives the Requirement Agent graph: run_chat_step, resume_approval_step, get_agent_state_details
app/services/design_service.py	Drives the Design Agent graph: start_design_generation, resume_design_approval, version history
app/services/development_service.py	Drives the Development orchestrator: start/approve/reject, tasks/agents/logs/artifacts accessors
app/services/llm_provider.py	LLM abstraction — wraps Google GenAI (Gemini) as a LangChain BaseChatModel, plus a MockChatModel fallback for 429/offline use
app/services/document_generator.py	Renders SRS/SDD to PDF (ReportLab), DOCX (python-docx), Markdown and JSON
app/services/document_parser.py	Extracts text from uploaded requirement documents
app/services/input_guardrails.py	Relevance check + prompt-injection detection for chat/upload input

Path	Responsibility
app/services/requirement_validator.py	Scoring / completeness rules used by the Requirement Agent's validation_node
app/services/json_repair.py	Repairs malformed JSON returned by the LLM before Pydantic parsing
app/prompts/*.py	Prompt templates for the Requirement and Design agents
alembic/	Database migrations

4. Data Model

All persistent state is stored via SQLAlchemy models in `app/models.py`. Each phase keeps its own versioned history table so prior SRS/SDD/code revisions are never lost, and each generation run is independently auditable.

Table	Purpose
projects	One row per SDLC project: name, description, current_phase, status
requirements / requirement_versions	Requirement doc container + numbered SRS JSON snapshots with reviewer comments
design_documents / design_versions	Design doc container + numbered SDD JSON snapshots, linked to the requirement version they were built from
development_versions	Numbered code-generation snapshots: manifest JSON, zip path, artifact paths, parent_version_id for revision chains
development_reviews	Human approve/reject decisions for a development version, incl. quality/security scores
development_tasks	Per-module task queue entries (owner agent, dependencies, priority, status)
development_logs / development_execution_logs	Free-text activity log + structured per-node execution telemetry (timing, token counts, retries)
development_agent_outputs	Raw per-agent output payloads (file name → source content) for each version
development_artifacts	Generated file pointers (source_zip, database_scripts, api_docs, project_report, change_log, ...)
epics / features / user_stories / acceptance_criteria	Optional backlog hierarchy synthesized from requirements (Stage 4 of the Requirement Agent)
project_agent_states	The live LangGraph checkpoint mirror for the Requirement Agent (current_state, document, messages, memory, missing_info)
human_reviews	Generic approve/reject audit trail keyed by phase + stage
activity_logs	Project-level activity feed shown in the dashboard
version_snapshots	Generic ProjectDocument snapshots used for rollback/versioning

5. Phase 1 — Requirement Agent Codeflow

Implemented in `app/agents/requirement_agent.py` as a LangGraph StateGraph over AgentState (`app/agents/state.py`). It is driven from the outside by `app/services/agent_service.py`, whose `run_chat_step()` and `resume_approval_step()` functions feed user messages / approval decisions into the graph and persist the resulting state onto `project_agent_states`.

5.1 Four-Stage Gated Pipeline

- Stage 1 — Extraction Review: the LLM (or MockChatModel fallback) extracts a RequirementMemory / ProjectDocument from the conversation; a human reviews the extracted content.
- Stage 2 — Gap Detection & Clarifications: `gap_detector_node` finds missing information; if gaps exist the graph loops back into `waiting_for_clarification`, otherwise the human approves the gap list.
- Stage 3 — Quality Validation Audit: `validation_node` scores the document (via `requirement_validator.py`); the pipeline requires a score ≥ 70 before continuing, retrying up to 3 times.
- Stage 4 — Backlog Synthesis & Finalization Sign-Off: `finalizer_node` compiles the final SRS and epics/features/user stories/acceptance criteria backlog; a final human sign-off hands the project to the Design Agent.

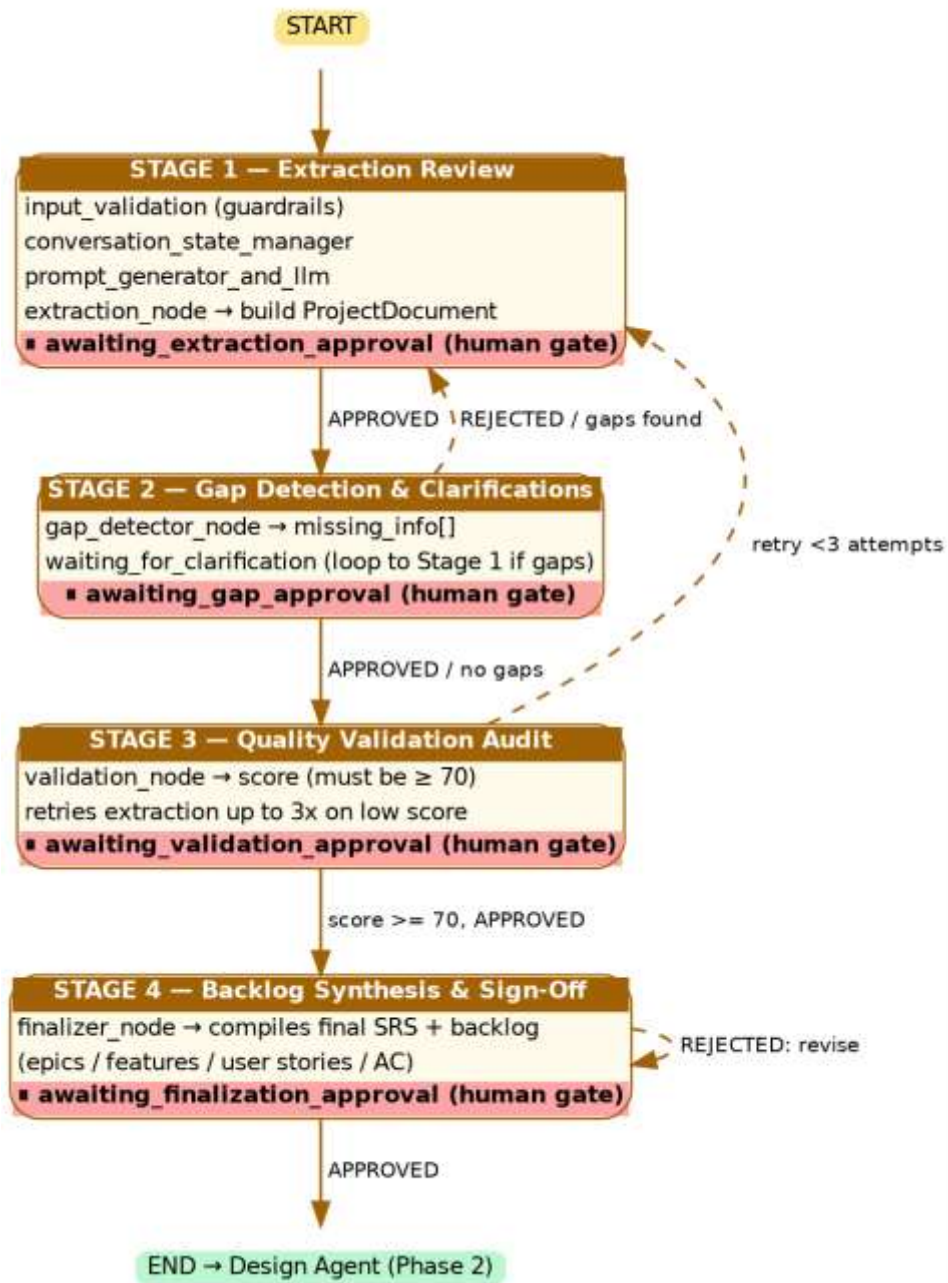


Figure 2 — Requirement Agent LangGraph: nodes, human-approval gates (red) and retry/reject loops (dashed).

5.2 Key Nodes

Node	What it does
input_validation	Runs guardrail checks (relevance + prompt-injection) on the incoming message
conversation_state_manager	Appends the message to state and decides the next conversational step
prompt_generator_and_llm	Builds the extraction prompt and calls the LLM provider
extraction_node	Parses the LLM's JSON into a RequirementMemory / ProjectDocument (with build_fallback_memory as a deterministic backstop)

Node	What it does
awaiting_extraction_approval	Human gate — pauses the graph (LangGraph interrupt) until Stage 1 is approved/rejected
gap_detector_node	Compares the document against completeness heuristics and lists missing_info
awaiting_gap_approval	Human gate for Stage 2
waiting_for_clarification	Loops back to the conversation manager to ask the user for missing details
validation_node	Computes a quality/completeness score; must be ≥ 70 to proceed
awaiting_validation_approval	Human gate for Stage 3
finalizer_node	Compiles the final SRS document and the epic/feature/story/AC backlog
awaiting_finalization_approval	Human gate for Stage 4 — approval flips project.current_phase to DESIGN

6. Phase 2 — Design Agent Codeflow

Implemented in `app/agents/design_agent.py` over `DesignAgentState` (`app/agents/design_state.py`), driven by `app/services/design_service.py`. `start_design_generation()` seeds the graph with the approved `ProjectDocument` from Phase 1 and runs it forward; `resume_design_approval()` handles the human gates.

6.1 Linear Design Pipeline (Stages 1-9)

Unlike the Requirement Agent, most of the Design Agent's work happens as an unconditional linear chain of specialist nodes, followed by three more human-gated review stages:

- `requirement_analyzer` → `architecture_planner` → `technology_selector` → `module_designer` → `database_designer` → `api_designer` → `security_designer` → `deployment_planner` → `diagram_generator` → `sdd_generator`
- Then: `awaiting_design_generation_approval` (human gate) → `design_gap_detector_node` → `awaiting_design_gap_approval` (human gate) → `design_validation_node` → `awaiting_design_validation_approval` (human gate) → `design_finalization_node` → `awaiting_design_finalization_approval` (human gate) → END.

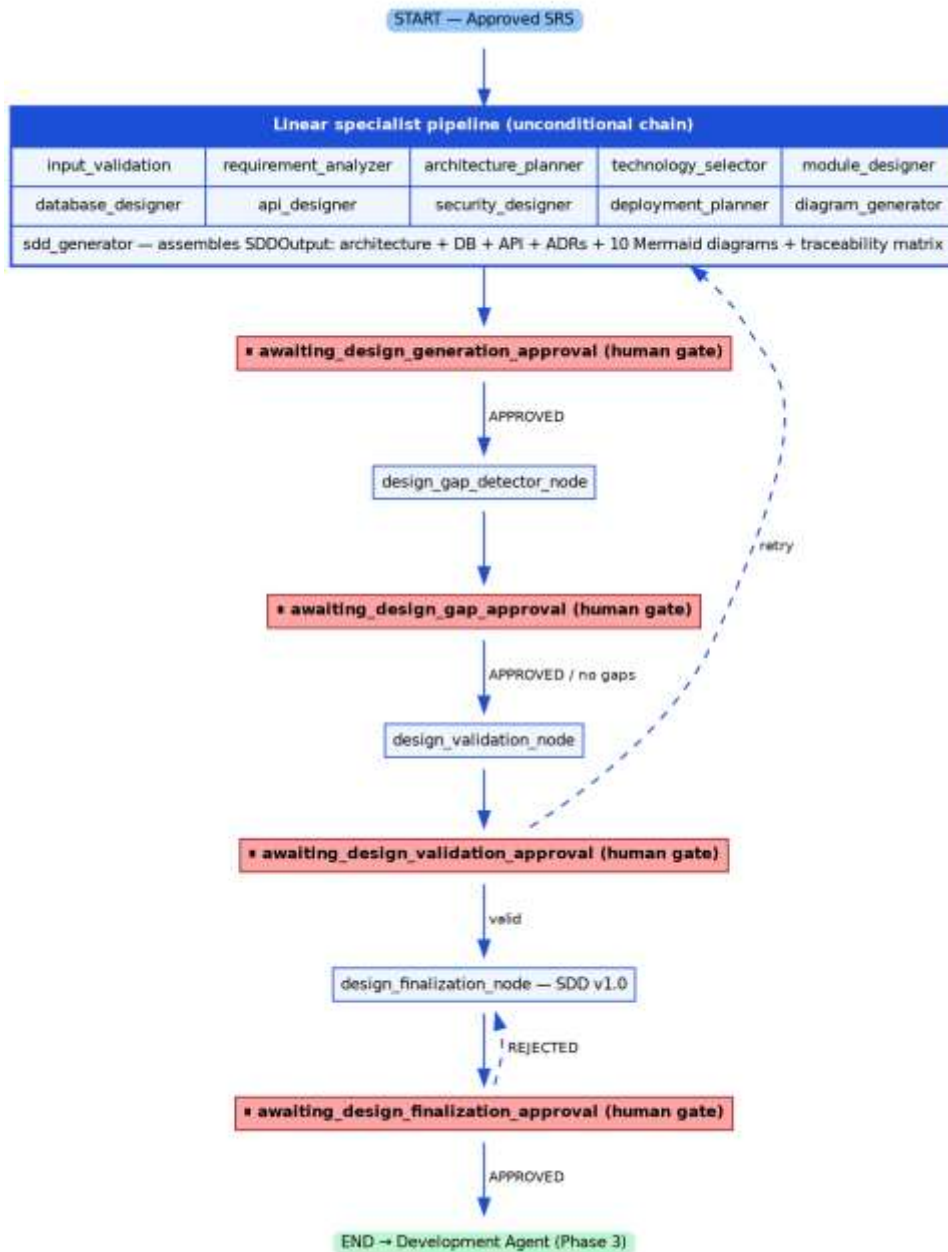


Figure 3 — Design Agent LangGraph: the specialist pipeline feeding `sdd_generator`, followed by three gated review stages.

6.2 What Each Specialist Produces

Node	Output
architecture_planner	Recommended architecture style (e.g. Modular Monolith or Event-Driven Microservices) with rationale
technology_selector	Concrete technology stack choices for the recommended architecture
module_designer	High/low-level module breakdown
database_designer	Table schemas, columns, types, PK/FK/unique constraints, engine selection (e.g. PostgreSQL 16 + Redis 7)
api_designer	REST endpoints, methods, payloads, response templates, error codes, JWT Bearer auth flow

Node	Output
security_designer	Security design policies, authorization flow
deployment_planner	Deployment/infrastructure topology across dev, staging, multi-AZ production
diagram_generator	10 Mermaid.js diagrams: C4 context, container/component, ER, sequence, deployment, class, activity, use-case, DB relationship, flow
sdd_generator	Assembles the full SDDOutput: ADRs (ADR-001..N) and the 100% requirement traceability matrix (REQ → module → API route → DB entity → ADR)

If no LLM-produced SDD is available, `build_fallback_sdd()` deterministically synthesizes a minimal-but-valid SDD from the approved requirement document, so the pipeline never dead-ends purely because of an LLM outage.

7. Phase 3 — Development Agent Codeflow

Implemented in `app/agents/development/orchestrator.py` over `DevelopmentAgentState` (`app/agents/development/development_state.py`), driven by `app/services/development_service.py`. This is the most complex graph: it fans work out to five specialist sub-agents in dependency order, merges their output, validates the build, and only then asks for human approval.

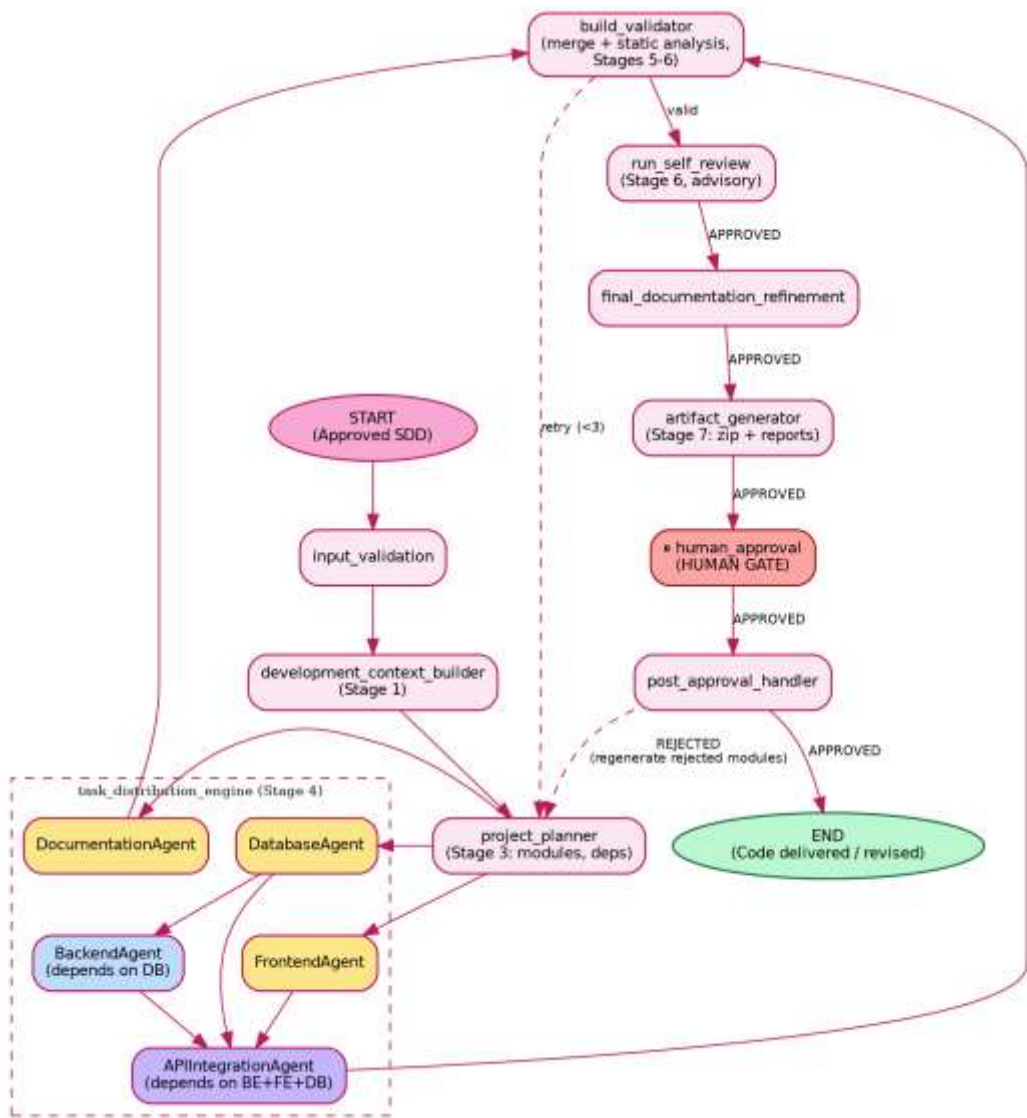


Figure 4 — Development Agent orchestrator: 9-stage pipeline with parallel/dependency-aware sub-agent scheduling inside `task_distribution_engine`.

7.1 Pipeline Stages

Stage	Node	Description
1	input_validation	Validates the incoming approved SRS/SDD payload before any generation starts
1	development_context_builder	Builds development_context (tech stack, approved SRS + SDD merged) via <code>context_builder.py</code>

Stage	Node	Description
3	project_planner	Uses planning_engine.py to derive modules, tasks and the task dependency graph
4	task_distribution_engine	Dispatches the 5 worker agents (see 7.2) in dependency-aware phases and merges validation_errors
5-6	build_validator	Merges generated files (core/merge_validation.py), runs static analysis; retries the plan up to 3 times on failure
6	run_self_review	Advisory self-review pass over the merged codebase (self_review_agent.py)
—	final_documentation_refinement	Polishes README/docs before packaging
7	artifact_generator	Packages the project into a downloadable zip + individual artifacts (core/artifact_generator.py)
—	human_approval	Human gate — approve, reject specific modules, or request revision
—	post_approval_handler	On REJECTED, feeds rejected_modules back into project_planner for a scoped regeneration; on APPROVED, finalizes the version

7.2 Task Distribution Engine — Parallel Sub-Agent Scheduling

task_distribution_engine() runs the five worker agents with a ThreadPoolExecutor, respecting their real dependencies:

- Phase 1 (parallel, 3 workers): DatabaseAgent, FrontendAgent and DocumentationAgent run concurrently — they don't depend on each other's output.
- Phase 2: BackendAgent runs next because it needs database_output (schema) from DatabaseAgent.
- Phase 3: APIIntegrationAgent runs last because it needs backend_output, frontend_output and database_output to reconcile routes/contracts and surface mismatches.

Each worker agent lives in app/agents/development/agents/ and returns a *_output dict plus any validation_errors, which are aggregated into agent_statuses (COMPLETED/FAILED) and an api_contract summary (backend_routes, mismatches).

Agent file	Role
database_agent.py	Generates schema/migration scripts and seed data from the SDD's database design
backend_agent.py	Generates backend (FastAPI/service) source files implementing the API design, consuming database_output
frontend_agent.py	Generates frontend source files implementing the UI per the SDD's module breakdown
api_integration_agent.py	Cross-checks backend routes against frontend API calls, reports mismatches, assembles the api_contract
documentation_agent.py	Generates project README / developer docs for the produced codebase
self_review_agent.py	Advisory pass: flags quality/security concerns without blocking the pipeline

7.3 Supporting Core Engines

Module	Purpose
core/rules_engine.py	Encodes coding-standard / generation rules the worker agents must follow
core/template_engine.py	Renders boilerplate/scaffolding templates for generated files
core/prompt_manager.py	Builds per-agent LLM prompts from development_context and task assignments
core/merge_validation.py	Merges each agent's generated_files into a single file tree and validates for conflicts
core/memory_manager.py	Lightweight memory helper shared across the development graph run
core/artifact_generator.py	Writes the merged file tree to disk and produces the downloadable zip + per-type artifact paths
core/code_snippet_store.py	Caches/retrieves reusable code snippets referenced by multiple agents

8. API Endpoint Reference

All endpoints are prefixed by the FastAPI app root and require the X-API-Key header (except /, /health, /docs, /openapi.json, /redoc).

8.1 Projects

Method	Path	Description
POST	/api/projects	Create a new SDLC project
GET	/api/projects	List all projects
GET	/api/projects/{id}	Get a single project
GET	/api/projects/{id}/status	Aggregated status: phase, SRS/SDD data, agent states, computed metrics, activity log
GET	/api/projects/{id}/history	SRS version history

8.2 Requirement Agent (Phase 1)

Method	Path	Description
POST	/api/projects/{id}/chat	Send a problem statement / chat message to the Requirement Agent
POST	/api/projects/{id}/requirements/upload	Upload a requirements document (parsed + guardrail-checked, then fed into chat)
GET	/api/projects/{id}/requirements/state	Current Requirement Agent state (memory, document, gaps, phase)
POST	/api/projects/{id}/requirements/approve	Approve the current stage (or fast-track to Design Agent)
POST	/api/projects/{id}/requirements/reject	Reject the current stage with reviewer comments
GET	/api/projects/{id}/requirements/download/{pdf docx markdown json}	Download the SRS in the given format

8.3 Design Agent (Phase 2)

Method	Path	Description
POST	/api/projects/{id}/design/generate	Trigger Design Agent generation from the approved SRS
GET	/api/projects/{id}/design/state	Current Design Agent state (phase, validation errors, gaps)
GET	/api/projects/{id}/design	Get the current design document + latest version
GET	/api/projects/{id}/design/versions	Design version history

Method	Path	Description
POST	/api/projects/{id}/design/approve	Approve a design stage / finalize SDD v1.0
POST	/api/projects/{id}/design/reject	Reject a design stage with reviewer comments
GET	/api/projects/{id}/design/download/{pdf docx markdown json}	Download the SDD in the given format

8.4 Development Agent (Phase 3)

Method	Path	Description
POST	/api/projects/{id}/development/start	Trigger code generation from the approved SDD
GET	/api/projects/{id}/development	Development state + version history
GET	/api/projects/{id}/development/tasks	Task queue for the current version
GET	/api/projects/{id}/development/agents	Per sub-agent status (COMPLETED/FAILED/PENDING)
GET	/api/projects/{id}/development/logs	Structured per-node execution logs
GET	/api/projects/{id}/development/files	Generated file listing
GET	/api/projects/{id}/development/artifacts	Generated artifact listing
POST	/api/projects/{id}/development/approve	Approve the generated code (optionally scoping to modules)
POST	/api/projects/{id}/development/reject	Reject and request full revision
POST	/api/projects/{id}/development/revise	Reject specific modules for scoped regeneration
GET	/api/projects/{id}/development/download	Download the full generated-code zip
GET	/api/projects/{id}/development/artifacts/download?type=...	Download a specific artifact (source_zip, database_scripts, api_docs, ...)

9. Frontend Architecture

The frontend is a single-page React + TypeScript app (frontend/src) built with Vite and styled with Tailwind CSS.

File	Responsibility
src/main.tsx	React root bootstrap
src/App.tsx	Main application shell: project dashboard, chat UI for the Requirement Agent, SRS/SDD viewers, Mermaid diagram rendering (incl. a <code>cleanMermaidChart()</code> sanitizer for malformed LLM-generated diagram syntax), approval actions, document downloads, version history
src/components/DevelopmentPanel.tsx	Phase 3 UI: task list, per-agent status, execution logs, generated files/artifacts, approve/reject/revise actions, zip download
src/services/api.ts	Typed fetch wrapper (<code>apiFetch/api</code>) that attaches the X-API-Key header and exposes typed methods for every backend route; also exports shared TypeScript types (<code>Project</code> , <code>SRSOutput</code> , ...)

The frontend initializes Mermaid.js at startup (dark theme) to render every `*_diagram_mermaid` field returned in the SDD. Because diagram text is LLM-generated, App.tsx includes defensive post-processing (`cleanMermaidChart`) that rewrites common malformed/PlantUML-flavoured snippets into valid Mermaid graph syntax before rendering.

10. End-to-End Walkthrough

A concrete trace through the whole system, showing which files are involved at each step:

#	Step	What happens in code
1	User creates a project	Frontend → POST /api/projects → project_service.create_project() → projects row inserted.
2	User sends a problem statement	Frontend → POST /api/projects/{id}/chat → agent_service.run_chat_step() invokes the compiled Requirement Agent graph (requirement_agent.py) with the checkpointier keyed by project_id; the graph runs until it hits a human interrupt or END.
3	Human reviews & approves each of the 4 SRS stages	Frontend → POST .../requirements/approve → agent_service.resume_approval_step() resumes the graph past the interrupt node; after Stage 4 approval, project.current_phase becomes DESIGN.
4	SRS is exported	Frontend → GET .../requirements/download/{fmt} → document_generator renders the stored SRS JSON to the requested format.
5	Design generation is triggered	Frontend → POST /api/projects/{id}/design/generate → design_service.start_design_generation() seeds DesignAgentState from the approved ProjectDocument and runs the Design Agent graph (design_agent.py) through its 9 specialist nodes to sdd_generator.
6	Human reviews & approves the 3 design gates	Frontend → POST .../design/approve (or /reject) → design_service.resume_design_approval() resumes the graph; final approval writes a design_versions row and flips project.current_phase to DEVELOPMENT.
7	Code generation is triggered	Frontend → POST /api/projects/{id}/development/start → development_service.start_development_generation() seeds DevelopmentAgentState with the approved SRS+SDD and runs orchestrator.py end to end (context → plan → parallel sub-agents → merge/validate → self-review → docs → artifact zip).
8	Human reviews the generated code	Frontend (DevelopmentPanel) polls .../development/tasks, /agents, /logs, /files, /artifacts, then POSTs .../development/approve (whole project) or .../development/revise (specific rejected_modules, which re-enters project_planner for a scoped regeneration).
9	Code is downloaded	Frontend → GET .../development/download (full zip) or .../development/artifacts/download?type=... (single artifact) → files streamed from disk paths stored on the DevelopmentVersion row.

11. Running the Project Locally

11.1 Backend

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# .env: GOOGLE_API_KEY / GEMINI_API_KEY, STUDIO_API_KEY, DATABASE_URL
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Backend runs at <http://127.0.0.1:8000>, interactive OpenAPI docs at </docs>.

11.2 Frontend

```
cd frontend
npm install
# .env: VITE_API_BASE=http://localhost:8000/api, VITE_STUDIO_API_KEY=...
npm run dev
```

Frontend runs at <http://localhost:3000> (or Vite's default 5173, both are already whitelisted in the backend's CORS config).

Appendix A — Source File Tree

Reverse-engineered from the uploaded archive (backend/app and frontend/src only; config/build files omitted):

```
backend/app/  
  main.py                # FastAPI app + all routes  
  database.py            # Engine, session, LangGraph checkpointer  
  models.py              # SQLAlchemy ORM models  
  schemas.py             # Pydantic schemas  
  agents/  
    state.py             # AgentState (Requirement Agent)  
    design_state.py      # DesignAgentState  
    development_state.py # (legacy) development state  
    requirement_agent.py # Phase 1 LangGraph graph  
    design_agent.py      # Phase 2 LangGraph graph  
    development_agent.py # (legacy/compat) development agent entry  
  development/  
    orchestrator.py      # Phase 3 LangGraph graph  
    planning_engine.py   # Module/task planning  
    development_state.py # DevelopmentAgentState  
    context_builder.py   # Builds development_context  
  agents/  
    database_agent.py  
    backend_agent.py  
    frontend_agent.py  
    api_integration_agent.py  
    documentation_agent.py  
    self_review_agent.py  
  core/  
    rules_engine.py  
    template_engine.py  
    prompt_manager.py  
    merge_validation.py  
    memory_manager.py  
    artifact_generator.py  
    code_snippet_store.py  
  services/  
    project_service.py  
    agent_service.py  
    design_service.py  
    development_service.py  
    llm_provider.py  
    document_generator.py  
    document_parser.py  
    input_guardrails.py  
    requirement_validator.py  
    json_repair.py  
  prompts/  
    requirement_prompts.py  
    design_prompts.py  
  alembic/               # DB migrations  
  
frontend/src/  
  main.tsx  
  App.tsx  
  components/  
    DevelopmentPanel.tsx  
  services/  
    api.ts
```